UNIVERSIDAD DE MANIZALES

FACULTAD DE CIENCIAS E INGENIERÍAS

DOCUMENTO DE DECISIONES DE DISEÑO

Evolución del Sistema de Gestión Académica

Actividad Colaborativa N.º 2 | Programación Orientada a Objetos

Presentado por

JUAN DAVID GÁLVEZ MOLINA

Ingeniería de Sistemas

Modalidad individual autorizada por el docente

2026

1. Introducción

El presente documento expone y fundamenta las decisiones de diseño adoptadas para evolucionar el Sistema de Gestión Académica desarrollado en la Actividad Colaborativa N.º 1 de Programación Orientada a Objetos. La evolución incorpora abstracción formal de personas, organización de grupos académicos, composición de horarios y excepciones personalizadas para operaciones de matrícula y consulta. Se mantiene la estructura funcional del proyecto original y se integran los nuevos componentes en el mismo programa ejecutable desde la terminal.

2. Descripción general de la solución

La implementación conserva en modelos.py las clases ProgramaAcademico, Asignatura, Persona, Estudiante, Docente y Administrativo, así como sus validaciones, constructores y métodos de visualización. Sobre esta base, grupo_academico.py introduce las clases GrupoAcademico y Horario; sistema_academico.py organiza el registro y la búsqueda de programas, asignaturas y grupos; y excepciones.py concentra las condiciones excepcionales del dominio. main.py integra en un único recorrido las demostraciones de la primera actividad y los escenarios exigidos en la segunda, mientras que test_sistema.py y test_actividad2.py comprueban la regresión y la nueva funcionalidad.

3. Generalización de personas mediante abstracción

La clase Persona se transformó en una clase abstracta utilizando ABC, del módulo estándar abc, y el decorador @abstractmethod sobre realizar_actividad_principal(). Esta modificación expresa una restricción del modelo: en la universidad no basta con registrar a una persona indeterminada, pues toda persona vinculada desempeña un rol concreto. Por ello, Persona conserva la identificación, el nombre, el correo y mostrar_informacion(), pero no puede construirse de forma independiente. Python impide la instanciación mientras exista un método abstracto sin implementar.

Estudiante, Docente y Administrativo mantienen su herencia desde Persona y ejecutan super().__init__() para reutilizar la inicialización y validación de los datos comunes. Cada subclase implementa realizar_actividad_principal(): el estudiante informa su programa y semestre; el docente señala su facultad; y el administrativo indica su cargo y dependencia. Asimismo, cada una conserva su versión particular de mostrar_informacion(), lo cual permite consultar sus datos específicos sin replicar la estructura general.

4. Organización de la oferta y relaciones entre objetos

GrupoAcademico recibe referencias a objetos ya construidos de Asignatura y Docente, no simples textos que representen sus nombres. Esta decisión garantiza que el grupo corresponda a entidades existentes y validadas por el sistema. El grupo también mantiene una colección privada de objetos Estudiante para administrar las matrículas mediante sus códigos estudiantiles. La clase SistemaAcademico centraliza tres catálogos: programas, asignaturas y grupos; mediante sus métodos registrar_programa(), registrar_asignatura(), registrar_grupo(), buscar_programa() y buscar_asignatura() evita reproducir búsquedas dispersas en el archivo principal.

Relación	Implementación	Sentido
GrupoAcadémico — Asignatura	Referencia a un objeto Asignatura existente	Una asignatura puede ofrecerse en varios grupos.
GrupoAcadémico — Docente	Referencia a un objeto Docente existente	El docente está asignado como responsable de un grupo.
GrupoAcadémico — Estudiante	Colección privada de objetos Estudiante	La matrícula incorpora o retira estudiantes del grupo.
GrupoAcadémico — Horario	Creación interna de un objeto Horario	Cada grupo posee su propio horario de operación.

5. Composición del horario y distinción frente a herencia

La relación entre GrupoAcademico y Horario es de composición porque el grupo construye internamente su propio objeto Horario durante __init__, a partir del día, hora inicial, hora final y salón. El horario queda como atributo privado del grupo, accesible mediante una propiedad de consulta. Dos grupos distintos pueden compartir la misma asignatura sin compartir el mismo horario: cada uno crea y conserva su propia instancia. La clase Horario valida que los valores horarios tengan formato HH:MM y que la hora final sea posterior a la inicial.

No es una relación de herencia: un grupo académico no es un tipo particular de horario, sino una organización académica que contiene un horario. Por consiguiente, no existe una relación “es un”, sino una relación “tiene un”. La implementación utiliza composición para representar una parte funcional del grupo, mientras que la herencia queda reservada a la especialización de Persona en sus tipos concretos.

6. Operaciones de matrícula, retiro y consulta

El método matricular_estudiante(estudiante) comprueba primero que el argumento sea un objeto Estudiante. Después verifica por codigo_estudiantil que el estudiante aún no esté registrado y, finalmente, valida que el número de matriculados sea inferior a capacidad_maxima. Esta secuencia separa dos fallas diferentes: intentar registrar un código repetido y tratar de ocupar un cupo inexistente. La colección de matrículas es privada y su propiedad de consulta devuelve una tupla, evitando modificaciones externas que eludan las validaciones del grupo.

El método retirar_estudiante(codigo_estudiantil) busca al estudiante por su código, lo retira cuando existe y lanza una excepción específica si no se encuentra. mostrar_estudiantes() devuelve el listado de estudiantes efectivamente matriculados y mostrar_informacion_grupo() reúne código del grupo, asignatura, docente, cupo, cantidad de matrículas y objeto horario. Estas operaciones se integran en el mismo sistema original, en lugar de desarrollarse como un programa independiente.

7. Excepciones personalizadas y continuidad

Las cinco situaciones excepcionales solicitadas se modelaron como clases concretas que heredan de ErrorAcademico, una clase base especializada de Exception. La distinción entre

excepciones permite reconocer con precisión el origen de cada anomalía. No se reemplazó el lanzamiento real de excepciones por mensajes condicionales: los métodos del dominio utilizan raise y el archivo main.py captura los errores mediante try/except para informar al usuario y continuar con los pasos siguientes.

Excepción personalizada	Origen del lanzamiento	Situación que controla
EstudianteYaMatriculadoError	GrupoAcademico.matricular_estudiante()	El código estudiantil ya pertenece al grupo.
CupoAgotadoError	GrupoAcademico.matricular_estudiante()	El número de matriculados alcanzó la capacidad máxima.
EstudianteNoMatriculadoError	GrupoAcademico.retirar_estudiante()	El código solicitado no figura entre los matriculados.
AsignaturaNoEncontradaError	SistemaAcademico.buscar_asignatura()	No se encuentra el código de asignatura.
ProgramaNoEncontradoError	SistemaAcademico.buscar_programa()	No se encuentra el código de programa.

En main.py, ejecutar_operacion() realiza cada acción dentro de un bloque try y captura explícitamente las cinco excepciones del dominio en except. Se muestran mensajes de error comprensibles y se prosigue con la siguiente operación. El recorrido termina consultando de nuevo los estudiantes matriculados, evidencia directa de que las anomalías controladas no interrumpen la ejecución general.

8. Conservación del polimorfismo de la Actividad N.º 1

La modificación de Persona no eliminó el polimorfismo previamente conseguido. El archivo main.py conserva una misma colección con dos objetos Estudiante, dos Docente y dos Administrativo. Un único ciclo recorre los seis elementos e invoca mostrar_informacion() y realizar_actividad_principal() sin identificar explícitamente el tipo de objeto mediante condicionales. Python resuelve automáticamente la implementación propia de la clase

concreta. La nueva abstracción refuerza este comportamiento, al exigir que toda subclase implemente su actividad principal.

9. Verificación funcional y control de versiones

La ejecución de main.py demuestra la imposibilidad de instanciar Persona, la creación de asignaturas, docentes, estudiantes, horarios y dos grupos, las matrículas correctas y los cinco escenarios excepcionales requeridos. La prueba termina con una consulta adicional que comprueba la continuidad del programa. La suite de pruebas automáticas se ejecutó mediante python -m unittest discover -v, con un resultado de 21 pruebas satisfactorias. Cinco corresponden a la regresión de la Actividad N.º 1 y dieciséis verifican la ampliación de la Actividad N.º 2.

La versión original está identificada en GitHub con la etiqueta actividad-1 y debe conservarse sin modificación. La evolución se entregará en el mismo repositorio SistemaGestionAcademica, mediante una actualización de la rama main y la etiqueta nueva y exacta Entrega_AC2. Esta separación por etiquetas identifica las dos entregas, conserva el historial y permite examinar el cambio progresivo sin sustituir el proyecto inicial.

10. Declaración de uso de herramientas de inteligencia artificial

Durante el desarrollo de esta actividad se utilizó ChatGPT, herramienta de inteligencia artificial de OpenAI, como apoyo para analizar los requisitos, proponer la evolución de las clases, revisar fragmentos de código Python, evaluar pruebas y corregir ortografía de la documentación técnica. La herramienta no sustituye la responsabilidad académica del estudiante de revisar, comprender, ejecutar, verificar y justificar la totalidad de las decisiones e implementaciones presentadas.

11. Conclusiones

La evolución del Sistema de Gestión Académica consolidó los mecanismos de especialización y polimorfismo desarrollados en la primera actividad, al sustituir la clase general instanciable por una abstracción que expresa los roles propios de la comunidad universitaria. El modelado de grupos académicos incorpora relaciones entre objetos reales y emplea composición para representar los horarios como componentes propios de cada grupo, sin confundir esta relación con la herencia.

A su vez, las operaciones de matrícula y retiro están controladas por validaciones y excepciones específicas, de manera que los errores previsibles no generan estados inconsistentes ni detienen la ejecución. La conservación de los mecanismos anteriores, la demostración funcional de los nuevos casos y las 21 pruebas automatizadas aprobadas sustentan la continuidad técnica de un único proyecto a través de las dos actividades.